A Dual-Multiplication-Mode and Reconfigurable Digital Compute-in-Memory Macro Using Precharge-Controlled 4T1C eDRAM

Ho-Sung Lee, Ik-Hyeon Jeon, Jiwon Lee, Eun-Bi Koh, and Joo-Hyung Chae, *Senior Member, IEEE*

***Abstract*—Recently, the demand for supporting diverse artificial intelligence (AI) operations within a single edge device has been increasing. However, existing compute-in-memory (CIM) architectures for edge AI are often optimized only for specific lightweight networks, limiting their adaptability. While digital CIM (DCIM) offers high accuracy and exhibits low sensitivity to variations in process, voltage, and temperature, their area and energy inefficiencies remain significant barriers to deployment in edge devices. This study proposes an embedded dynamic random access memory (eDRAM)-based dual-multiplication-mode (DMM) DCIM macro leveraging precharge-controlled 4T1C gain cells. The proposed design enables the AND/XNOR dual-multiplication mode within a 4T1C cell through controlled precharge states. It supports a wide range of neural networks for edge AI, including INT1–8 operations and binary neural networks, such as XNOR-net. An area-efficient adder tree structure reduced the transistor count by 19% compared to conventional adder trees. This structure also accommodates 1–8-bit signed and unsigned inputs and weights without wasting cells, enhancing reconfigurability. Fabricated using a 28-nm CMOS process, the 16-kb DMM-DCIM prototype chip demonstrated area and energy efficiencies of 28.9 TOPS/mm^2 and 190.3 TOPS/W for 4b–4b multiply-accumulate operations, respectively.**

***Index Terms*—Adder tree, compute-in-memory (CIM), edge artificial intelligence (AI), embedded dynamic random access memory (eDRAM), multiply-accumulate (MAC), precharge.**

I. Introduction

In VON Neumann architectures, limited data bandwidth between processor and memory creates a critical bottleneck, impeding performance improvements in artificial intelligence (AI) technologies [1]. Compute-in-memory (CIM) architectures address this issue by performing multiply-accumulate (MAC) operations directly within memory, enhancing computational efficiency while significantly reducing the power consumption and latency caused by data movement between processor and memory.

Received 30 January 2025; revised 26 March 2025; accepted 16 April 2025. Date of publication 15 May 2025; date of current version 29 September 2025. This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by Korean Government (MSIT) under Grant RS-2022-II221013 and in part by the National Research Foundation of Korea (NRF) Grant funded by Korean Government (MSIT) under Grant RS-2024-00334247 and Grant RS-2024-00414230. *(Corresponding author: Joo-Hyung Chae.)*

The authors are with the Department of Electronics and Communications Engineering, Kwangwoon University, Seoul 01897, South Korea (e-mail: jhchae@kw.ac.kr).

Digital Object Identifier 10.1109/TVLSI.2025.3562568

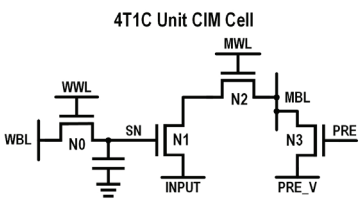
The various memory technologies applied in CIM designs include embedded nonvolatile magnetic memory [2], [3], resistive memory [4], [5], dynamic random access memory (DRAM) [6], [7], static RAM (SRAM) [8], [9], and embedded DRAM (eDRAM) [10], [11], [12], [13], [14]. Embedded nonvolatile magnetic and resistive memories are not suitable for CNNs that require fast and frequent model updates due to poor endurance and lower speeds. Similarly, DRAM primarily adopts a compute-near-memory design due to constraints, such as cell reliability and conventional memory operation support, including cost-sensitive processes and dense floorplans, which limit its throughput advantages [11]. The SRAM has the disadvantage of being less efficient than DRAM in terms of memory density, which limits its applicability for large CIM designs. In contrast, the eDRAM, composed of high-performance logic transistors, offers high-density embedded bit cells, low pJ/bit access energy, high endurance, and high bandwidth, making it an appropriate choice for CNN computations.

CIM solutions in edge devices must prioritize both area and energy efficiencies. Simultaneously, achieving sufficient accuracy is essential to deliver reliable and meaningful user experiences. Therefore, the specific computational scheme, either lightweight neural networks or medium-precision MAC operations, is commonly implemented in CIM for edge devices, which minimizes power consumption, area requirements, and storage overhead. CIM macros designed for binary neural networks, such as XNOR- and XOR-net, exemplify these approaches [8], [9], [15]. However, these solutions have the disadvantage of limited flexibility due to restricted algorithm compatibility, limited bit precision, and nonprogrammable architecture. As AI technologies rapidly advance and the functionality of edge devices expands, the need to support diverse AI operations within a single CIM architecture is growing. To meet these demands, a recent study [16] has improved the programmability of the CIM architecture through an instruction set architecture (ISA) and compiler design; the ISA acts as an interface between the hardware and the software, which can employ various operations and constraints to a compiler by using a programmable accelerator.

CIM architectures can be classified into analog CIM (ACIM) and digital DIM (DCIM) based on their MAC operation methods. ACIM offers superior energy and area efficiencies [17], [18], [19]. However, its accuracy and yield are constrained by circuit nonlinearity and high sensitivity

1063-8210 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.



(a)

AND Operation

SN	INPUT	MBL (Precharge)	MBL (AND Result)
0	0	0	0
0	VDD	0	0
VDD	0	0	0
VDD	VDD	0	VDD-V_{TH}

XNOR Operation

SN	INPUT	MBL (Precharge)	MBL (XNOR Result)
0	0	VDD	VDD
0	VDD	0	0
VDD	0	VDD	0
VDD	VDD	0	VDD-V_{TH}

(b)

Fig. 1. (a) Schematic of the precharge-controlled 4T1C GC eDRAM cell. (b) Truth tables for operations in DMM based on precharge value.

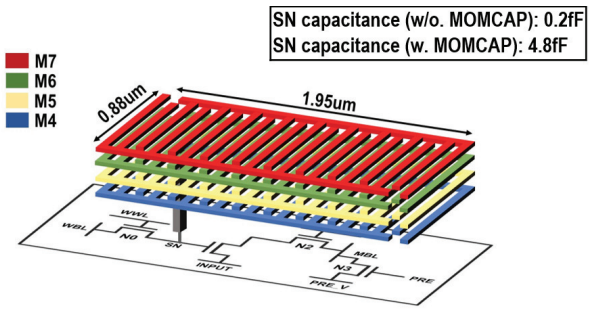
to process, voltage, and temperature variations. Conversely, DCIM ensures high accuracy with large-scale parallel computations [20], [21], [22]. However, improvements in energy and area efficiencies are essential for the widespread adoption in edge devices.

Hence, we propose an eDRAM-based dual-multiplication-mode (DMM) DCIM macro that uses precharge state control. The proposed eDRAM-based DCIM macro has three key features: 1) a novel precharge-controlled 4T1C gain cell (GC) with a customized metal-oxide–metal capacitor (MOMCAP) that improves data retention time (DRT) without expanding the cell footprint; 2) supporting AND and XNOR dual-multiplication operations with minimal additional logic components through the MAC bitline (MBL) precharge control mechanism; and 3) an area-efficient adder (AEA) tree structure [20], [21] reduces the transistor count by 19% compared to prior designs [22]. This adder structure also supports reconfigurable bit widths and accommodates both signed and unsigned input and weight values, enhancing versatility. The DMM-DCIM macro can support various edge AI operations, including XNOR-net and INT1–8 computations.

The remainder of this article is structured as follows. Section II introduces the DMM based on precharge state control. Section III details the architecture, implementation, and operation of the proposed DMM-DCIM macro. Section IV presents the measurement results. Finally, Section V provides the conclusions.

II. Precharge-Controlled Dual-Multiplying 4T1C eDRAM Cells

Fig. 1(a) depicts the precharge-controlled 4T1C GC unit. The CIM cell supports not only conventional write and precharge operations but also performs AND and XNOR logic operations based on the state of the MBL during the precharge phase [see Fig. 1(b)]. The MBL node is precharged to "0" in the AND mode, while the MBL node is precharged to the complement of the INPUT signal in the XNOR mode. Weight word line (WWL), MAC word line (MWL), and precharge (PRE) signals are driven to a voltage level higher than VDD to transfer logic "1" through NMOS switches (N0, N2, and N3) without incurring a threshold voltage (V_{TH}) loss. However, when logic "1" is stored in the storage node (SN), V_{TH} loss occurs at switch N1. Therefore, a low-V_{TH} transistor is employed for N1, ensuring that the MBL retains a voltage of VDD–V_{TH} as 853–951 mV for 1-V VDD voltage with the worst and best cases of V_{TH} loss. Furthermore, we used the low-VDD (LVDD) power domain less than worst case



DRT (Stored value)	Before Adding MOMCAP	After Adding MOMCAP
*Worst DRT (VDD)	3.28 us	211 us
**Worst DRT (0V)	0.751 us	57.4 us
***Average DRT	2.02 us	134.2 us

*: Simulated PVT condition = FF, 0.9V, -40℃
**: Simulated PVT condition = FF, 1.1V, 125℃
***: (Worst-case DRT for VDD + worst-case DRT for 0V) / 2

Fig. 2. Customized MOMCAP structure integrated with the eDRAM cell and simulated worst case DRT.

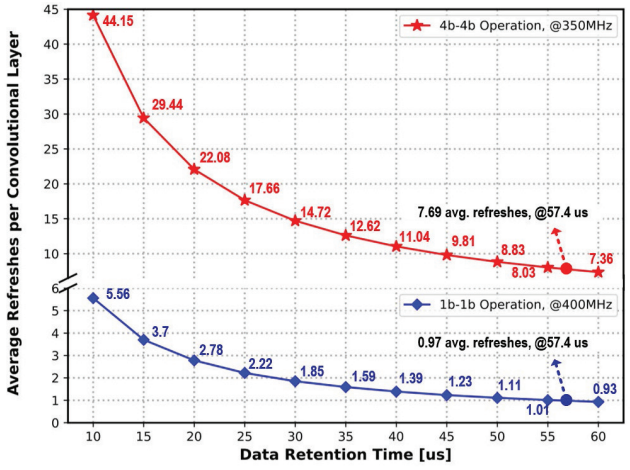
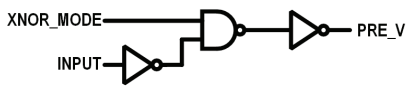


Fig. 3. Study on average refreshes per convolutional layer according to DRT.

VDD–V_{TH} in the digital adder tree to compensate for the V_{TH} drop of the input multiplied data and improve energy efficiency.

The eDRAM cell stores data within the parasitic capacitor of the SN, resulting in a reduced DRT. This limitation was addressed by incorporating a customized MOMCAP into the SN. Fig. 2 illustrates the MOMCAP structure, where four metal layers (metals 4–7) are stacked to maximize surface area. The MOMCAP was designed not to exceed the eDRAM cell layout area. Compared to the case without the MOMCAP, the SN capacitance with the MOMCAP was increased by 24 times. This increases the DRT while reducing the refresh energy without expanding the cell footprint. Simulated DRT results (see Fig. 2) indicate that the minimum DRT for storing VDD and 0 V after integrating the MOMCAP was 211 and 57.4 μs. Accordingly, the refresh period was configured to be shorter than this value. Similar to [23], Fig. 3 shows that in the worst case DRT of 57.4 μs, 0.97 and 7.69 average refreshes per convolutional layer are required for ResNet-50 at ImageNet in 1b–1b and 4b–4b computation, respectively. This

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.



(a)

XNOR_MODE	PRE_V
0	0
1	$\overline{\text{INPUT}}$

(b)

Fig. 4. (a) Block diagram and (b) truth table of PRE_V driver.

demonstrates that the DMM-DCIM macro offers high utility for architectural and system-level implementations. Similar to the SN, the precharge voltage is also stored in the parasitic capacitor of the MBL; thus, the MBL voltage reduces over time owing to the leakage current. The simulated worst DRT in the MBL was 37.4 μs. Since the DMM-DCIM macro requires only five clock cycles for 4b–4b MAC calculation, as described in Section III-D, our macro operates without problems.

Fig. 4(a) presents the block diagram of the PRE_V driver responsible for precharging. To alleviate the area overhead problem, each PRE_V driver was placed to share a single driver per row. Fig. 4(b) shows the truth table of the PRE_V driver's operation. When the XNOR_MODE signal was set to "0" (AND mode), the output signal, PRE_V, remained "0," irrespective of the INPUT value. Conversely, the PRE_V signal mirrored the inverted INPUT value under the XNOR mode (with the XNOR_MODE signal set to "1").

Fig. 5(a) outlines the three-step operation flow of the precharge-controlled 4T1C unit CIM cell. First, the weight stored in the weight bitline (WBL) was transferred to the SN by activating the N0 switch transistor. As described earlier, the WWL was driven at a voltage level higher than VDD, ensuring the storage of "1" without incurring V_{TH} loss. Second, the MBL was precharged to a value determined by the PRE_V driver, depending on whether the operation mode was AND or XNOR. Finally, the third step involved applying the INPUT value to the source of the N1 transistor, and the input and weight were multiplied through AND or XNOR operations based on the predefined truth table. Simultaneously, the N2 transistor transmitted the multiplication results to the MBL. Fig. 5(b) and (c) shows the simulated waveforms in the XNOR operation, which presents output logically "1" (VDD–V_{TH}) and "0" (0 V), respectively [see Fig. 1(b)]. In order to keep the SN data stable, a timing delay (T_D) was given between the moment when the WWL turned off and when the WBL precharged to half of VDD. This design enabled both AND and XNOR multiplication operations to be executed within a single 4T1C cell, thereby reducing the transistor count per cell.

III. 16-KB DMM- DIM Macro Architecture

A. Overall Architecture

Fig. 6 shows the overall architecture of the 16-kb eDRAM-based DMM-DCIM macro. The proposed DMM-DCIM macro comprised four 64 × 64 eDRAM-based CIM subarrays, each containing 16 global MAC units (GMUs). Each GMU integrated a global adder, an accumulator, and a local MAC unit (LMU) composed of four 64 × 1 bit cell arrays and local adders. The eDRAM cells used a 4T1C GC structure. The word line (WL), PRE_V driver, and in/out (I/O) peripheral circuits were situated outside the memory array, along with the CIM controller, input buffer, and weight buffer. The CIM controller managed command signals and data inputs for the DMM-DCIM macro, such as row addresses, input values, and weight data. Input and weight buffers are crucial in storing data required for MAC operations due to the limited number of I/O pads available in the chip design.

The DMM-DCIM macro supports both single-bit and multibit MAC operations. In single-bit mode, computations primarily involve the eDRAM cells and local adders, while the global adder and accumulator remain inactive. The local adders sum the multiplied outputs from the eDRAM cells, producing the local MAC (LMAC[27:0]) value. In the multibit mode, the global adder and accumulator complement the operations of the local adder. The global adder generates partial sum (PSUM[9:0]) by aggregating local adder outputs considering the digit positions of the weights. After that, the accumulator sums all PSUM values from the global adder aligned with the input digit positions, yielding the global MAC (GMAC[13:0]) output.

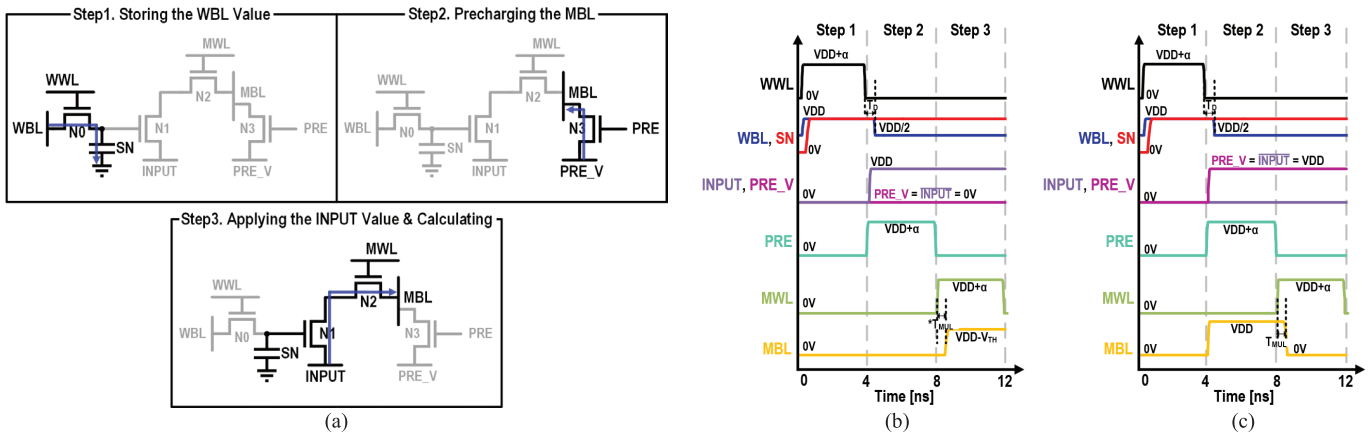
B. Area-Efficient Local Adder

We adopted an AEA to minimize area and energy overhead. Fig. 7 presents the block diagram of the local adder, comprising four 15:4 AEAs [24], two 4-bit ripple-carry adders (RCAs), one 5-bit RCA, and one 6-bit RCA. Among the various AEA ratio options (7:3 AEA, 15:4 AEA, 31:5 AEA, and 63:6 AEA), we selected the 15:4 AEA for the local adder design, considering the required transistor count and design complexity. The 15:4 AEA reduces the number of half adders (HAs) by compressing 15 1-bit inputs. Compared with the structure solely relying on the RCA with 63 HAs, integrating the AEA at the front end of the local adder lowers the HA count to six, significantly optimizing resource utilization. In particular, the 15:4 AEA, as shown in Fig. 8, reduces the number of full adders (FAs) and HAs by 1 and 4, respectively, compared to the 15:4 RCA structure, resulting in a total reduction of four FAs and 16 HAs in a single computation unit. Single-bit AND and XNOR operations share the local adder used in multibit operations. Thus, the macro area can be utilized efficiently because no additional circuit is needed to implement both AND and XNOR operations within the single unit.

C. Architecture of Digital Adder Tree

Fig. 9 presents the overall block diagram of the digital adder tree, which includes the local adder, global adder, accumulator, and output stage. The local adder summed the results of bitwise multiplication for 64 weights and 64 inputs in each column, while the global adder aggregated the outputs from four local adders, considering the digit positions of the weights [21]. The digital adder tree operated within a dedicated power domain, LVDD, with a supply voltage level approximating VDD–V_{TH}, enhancing energy efficiency. The global adder comprised two 8-bit RCAs, one 10-bit RCA, seven inverters, and several 2:1 multiplexers controlled by the Weight_Signed signal to manage signed weights. It functioned by shifting and summing the outputs from the local adders based on the digit positions of the weights. Conventional adder

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.



*: Time delay for multiplying in precharge-controlled 4T1C eDRAM cell

Fig. 5. (a) Three-step operation flow of the precharge-controlled 4T1C eDRAM cell and simulated waveforms in XNOR operation for (b) weight:1/input:1 and (c) weight: 1/input: 0.

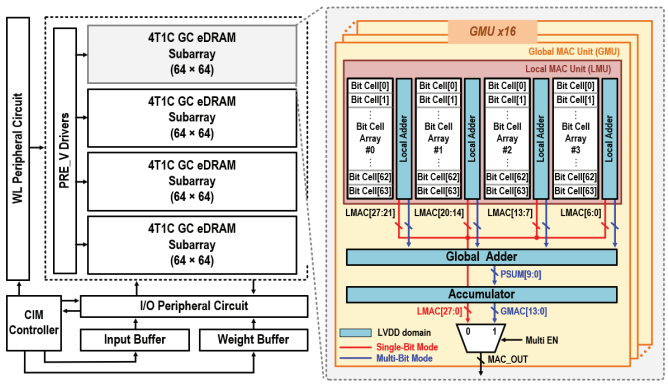


Fig. 6. Overall architecture of the proposed precharge-controlled 4T1C eDRAM-based DMM-DCIM macro.

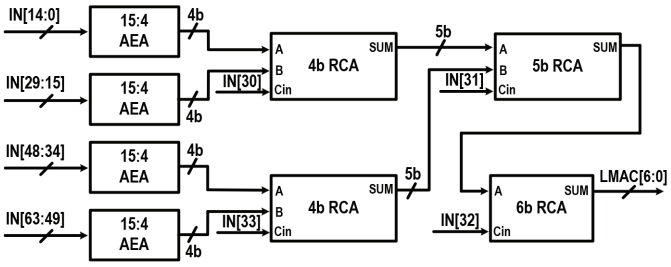


Fig. 7. Block diagram of the area-efficient local adder employing a 15:4 AEA.

tree designs rely on additional sign-extension (SE) circuits (18 transistors per RCA) to support signed weights [22]. In contrast, the proposed digital adder tree is designed so that each of the four local adders sums the 64 multiplication results of the 1-bit inputs and respective MSB, MSB–1, MSB–2, and LSB bits of the weights. Therefore, by using only seven 2:1 multiplexers and inverters, when the weight is a signed bit, the Weight_Signed signal is activated to 1, performing the $\times -1$ operation by inverting the output of the local adder that sums the multiplication results of the MSB of the weight and the input. This inverted value is then summed with the output

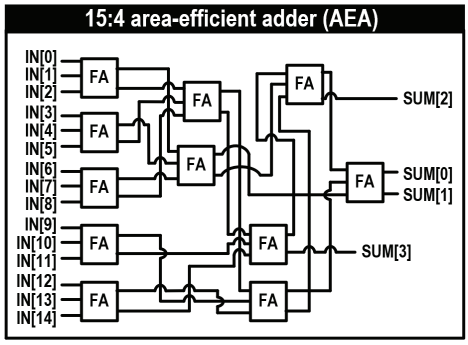


Fig. 8. Circuit diagram of 15:4 AEA block.

from another local adder, thereby supporting the signed bit of the weight [21]. Hence, this design improves the area efficiency of the DCIM macro by 19% compared to conventional adder trees without compromising accuracy. The DMM-DCIM macro processes inputs in a bit-serial manner, with the MSB transmitted first. The accumulator shifts the PSUM by one bit per cycle, stores it, and adds it to the output of the subsequent cycle. When adding signals A and B in the 14-bit (14b) adder of the accumulator, an SE operation must be applied to signal A to match the bit width of signal B. Therefore, before the PSUM[9:0] signal enters the 14b adder, it is extended by setting the MSB to PSUM [9] if the weight is a signed bit or to 0 if it is unsigned. As a result, signal A is extended to 11 bits. When the input is a signed bit, the Input_Signed signal is activated to 1 at the timing when the result of the global adder, which sums the 64 multiplication results of the input MSB bit and the 4-bit weight, is stored in the accumulator. The activated Input_Signed signal controls the number of 11 2:1 multiplexers and inverters, inverting the output of the global adder and thereby supporting signed input operations. Subsequently, an SE operation is performed, converting the 11-bit signal A to a 14-bit signal.

Fig. 10 compares the digital adder tree designs in [20], [24], and [22] with the proposed DCIM macro. The structure in [20] and [24] [see Fig. 10(a)] supports only 1-bit weight operations, whereas the proposed design supports various weight-bit

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.

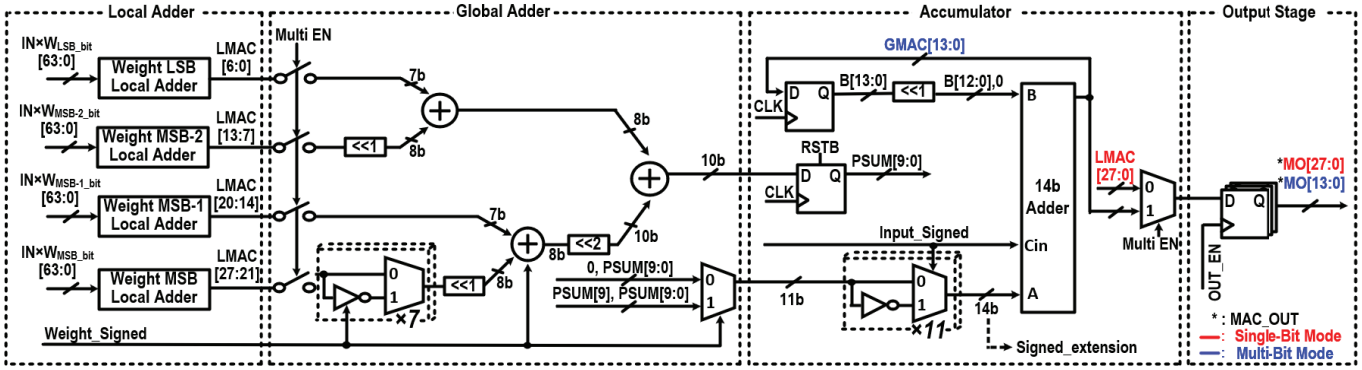


Fig. 9. Overall block diagram of the digital adder tree, incorporating the local adder, global adder, accumulator, and output stage.

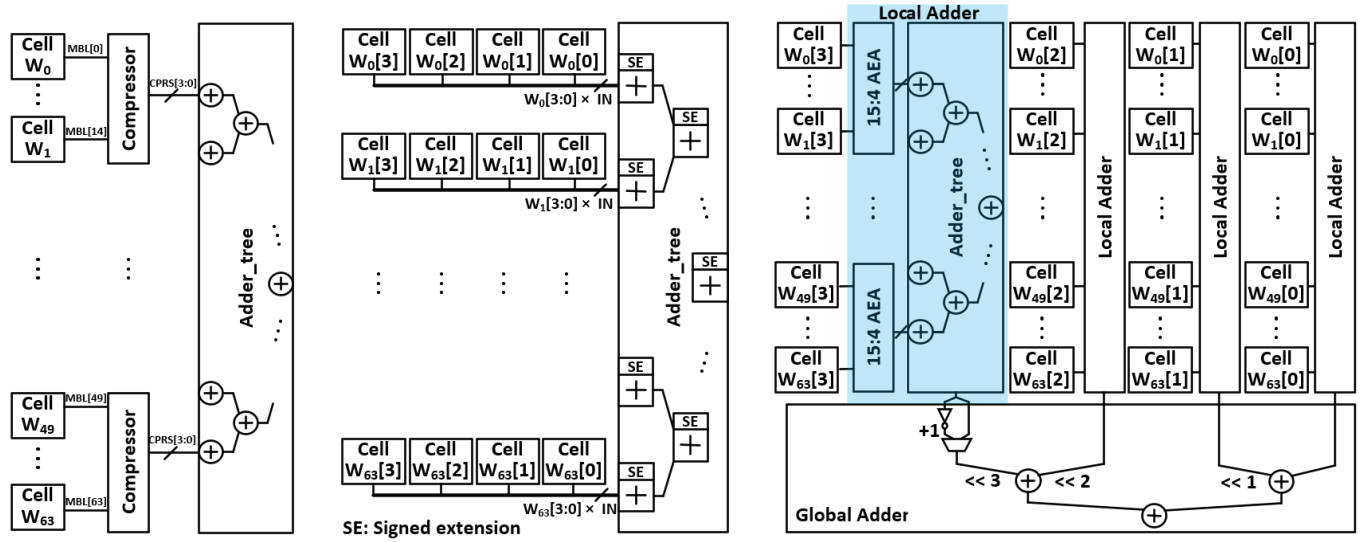


Fig. 10. Comparison of digital adder tree in (a) [20], [24], and (b) [22] with (c) proposed DCIM macro.

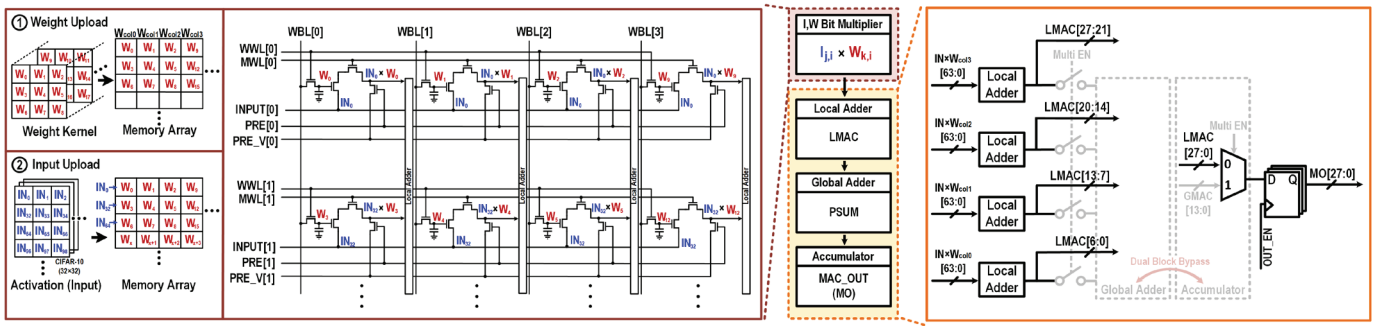


Fig. 11. Process of the 1b–1b AND/XNOR operation within the macro.

precision computations. Although the structure in [22] [see Fig. 10(b)] accommodates multibit weight operations, each RCA requires an SE circuit to execute signed-bit operations, increasing the area overhead. Additionally, the entire adder tree must be operated even for 1-bit computations, resulting in unnecessary consumption of power, area, and latency. In contrast, the proposed structure [see Fig. 10(c)] supports multibit weight operations while utilizing the local adder without modification. Furthermore, it executes signed weight-bit operations by summing the products of weight MSBs (the sign bits of weight) and the input values using the local adder. Subsequently, it performs the $\times -1$ operation with an inverter, eliminating the need for additional SE circuits. Therefore, this approach improves the area efficiency of the DCIM macro by 19% compared to the previous adder tree in [22] without sacrificing accuracy.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.

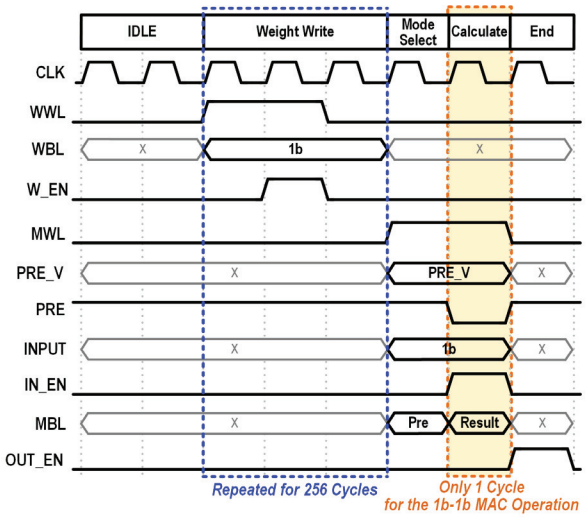


Fig. 12. Timing diagram of 1b–1b AND/XNOR operation.

D. Various MAC Operation Support

Fig. 11 shows the detailed MAC operations of the DMM-DCIM macro, specifically the 1b–1b AND and XNOR modes. In these modes, AND or XNOR operations are performed between 1-bit input and 1-bit weight. These operations occur simultaneously across 64 rows within a single column. Once the 64 multiplication results from each row are computed, they are fed into the local adder, which sums them together. The outcome of this process is the output of LMAC[27:0] from each GMU. This entire operation, from the initial AND/XNOR computation to the final addition and output, is completed in just one cycle.

Fig. 12 shows the timing diagram for the 1b–1b AND/XNOR mode operations. The operation begins with the "IDLE" phase, during which the WBL is precharged. The next phase is the "Weight Write" phase, where the weight values are written into the cells. This phase can be subdivided into three distinct subphases for precise control. In the first subphase, the WWL is activated, preparing the system to store the weight in the SN node, while the weight value is simultaneously written into the WBL. In the second subphase, the weight value entered into the WBL is stored in the SN node by activating the weight_enable (W_EN). In the final subphase, both WWL and W_EN are deactivated, indicating the completion of the weight writing process. As 64 weight data values are written simultaneously across all columns of a single row in each "Weight Write" cycle, this operation is repeated over 256 cycles, during which each weight value is sequentially stored in its corresponding row. Subsequently, the process enters the "Mode Select" phase, during which the multiplication mode is determined. In this phase, the MWL is activated, and the MBL is precharged according to the precharge (PRE) and PRE_V signals. The next phase is the "Calculate" phase, during which the actual MAC operation occurs. The input_enable (IN_EN) signal is activated, allowing the input value to be applied to the cells. The AND or XNOR operation is performed between the input value and the stored weights, and the result is transmitted to the MBL. The calculation ends with the deactivation of the MWL, and the output_enable (OUT_EN) is activated, making the result output in the final phase, called the "End" phase.

Fig. 13 illustrates the process of the 4b–4b AND operation within the macro, where 64 4-bit inputs are fed into the CIM array in a bit-serial manner. In the 4b–4b AND mode, when the weight is a signed bit, the Weight_Signed signal is activated to 1, which inverts the output of the local adder that sums the multiplication results of the MSB of the weight and the input. This inverted value is then summed with the output from another local adder. In contrast, when the weight is an unsigned bit, the output of the local adder is directly summed with the output from another local adder. The output of the global adder was calculated for unsigned weights as follows: $\text{PSUM} = 2^3 \times \Sigma(I_{j,i} \times W_{k=3,i}) + 2^2 \times \Sigma(I_{j,i} \times W_{k=2,i}) + 2^1 \times \Sigma(I_{j,i} \times W_{k=1,i}) + 2^0 \times \Sigma(I_{j,i} \times W_{k=0,i})$, while it was calculated for the signed weights as follows: $\text{PSUM} = -2^3 \times \Sigma(I_{j,i} \times W_{k=3,i}) + 2^2 \times \Sigma(I_{j,i} \times W_{k=2,i}) + 2^1 \times \Sigma(I_{j,i} \times W_{k=1,i}) + 2^0 \times \Sigma(I_{j,i} \times W_{k=0,i})$, where i denotes the row index, j represents the digit positions of the input, and k corresponds to the digit positions of the weight. The accumulator summed the PSUM values across multiple cycles while considering the input positions. The global adder computed partial sums during the first four cycles, while the accumulator aggregated these partial sums from the second through the fifth cycles in a pipelining manner. When the input is a signed bit, the Input_Signed signal is activated to 1, inverting the value obtained by summing the multiplication results of the weight and the input's MSB. This inverted value is then added to the outputs of the global adder in another cycle. In contrast, when the weight is an unsigned bit, the summed value of the weight and the input's MSB multiplication results are not inverted and are directly added to the outputs of the global adder calculated in another cycle. The accumulator output for unsigned inputs was calculated as: $\text{MAC_OUT} = 2^3 \times \text{PSUM}_{z=1} + 2^2 \times \text{PSUM}_{z=2} + 2^1 \times \text{PSUM}_{z=3} + 2^0 \times \text{PSUM}_{z=4}$, and the accumulator output for signed inputs was calculated as: $\text{MAC_OUT} = -2^3 \times \text{PSUM}_{z=1} + 2^2 \times \text{PSUM}_{z=2} + 2^1 \times \text{PSUM}_{z=3} + 2^0 \times \text{PSUM}_{z=4}$, where z denotes the cycle index.

Fig. 14 shows the timing diagram for the 4b–4b AND-mode operation. As previously described in the 1b–1b operation, the "IDLE" and "Weight Write" phases follow the same process in configuring the timing diagram. There is a difference between the 1b–1b operation and the number of cycles in the "Calculate" phase. In the "Mode Select" phase, the 4b–4b operation corresponds to an AND operation, so the MBL is precharged based on the PRE and PRE_V signals. In the "Calculate" phase, the IN_EN signal is activated, allowing the input value to be loaded into the cell. The MAC operation proceeds over four cycles, with the calculated values added through the adder in each cycle. After the MWL is turned off, one more cycle is required to complete five cycles in total because the final MAC operation value needs to move to the accumulator for the last calculation. After that, OUT_EN is activated, and the result is output in the final phase called the "End" phase.

This DMM-DCIM macro also facilitated extended bit-width MAC operations using external logic. An 8b–8b MAC operation can be executed as follows. First, a 4-bit portion of the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.

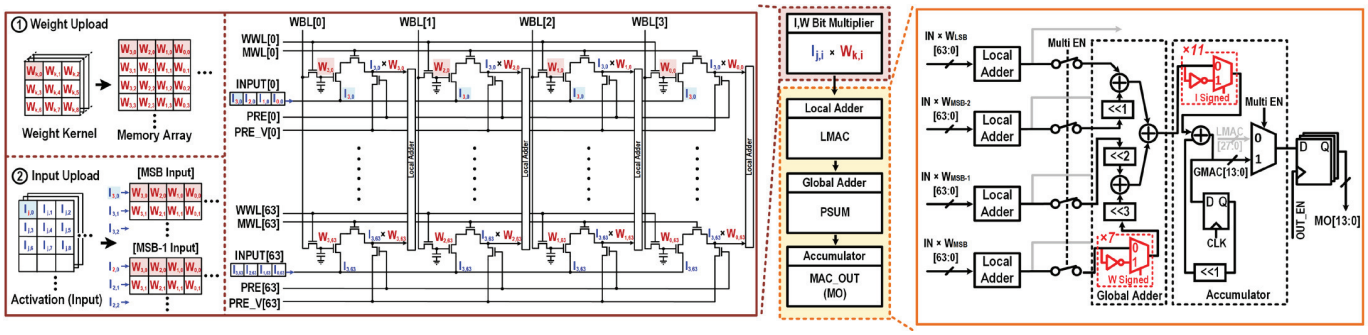


Fig. 13. Execution process for the 4b–4b AND operation within the macro.

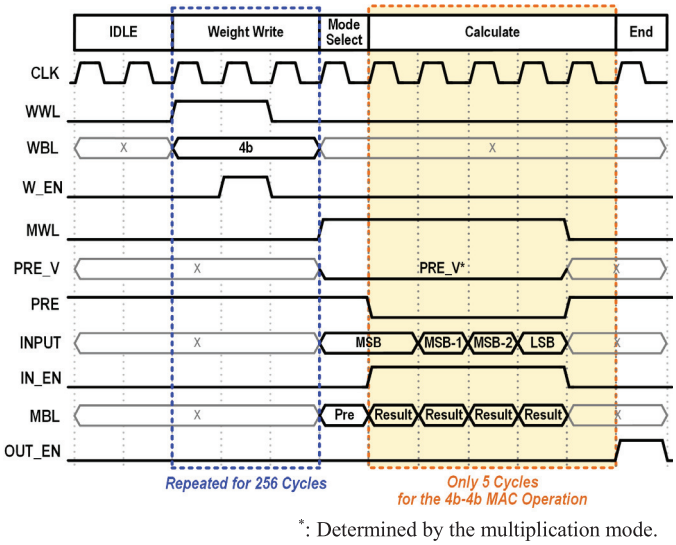


Fig. 14. Timing diagram of 4b–4b AND operation.

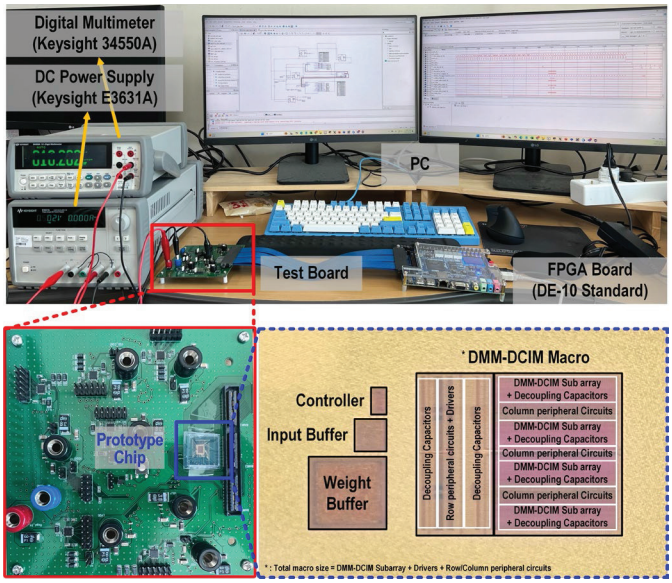


Fig. 15. Measurement setup and die micrograph of the prototype chip.

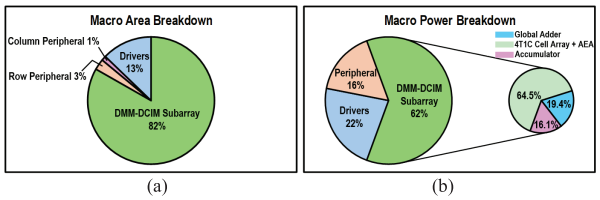


Fig. 16. (a) Area and (b) power breakdowns for DMM-DCIM macro.

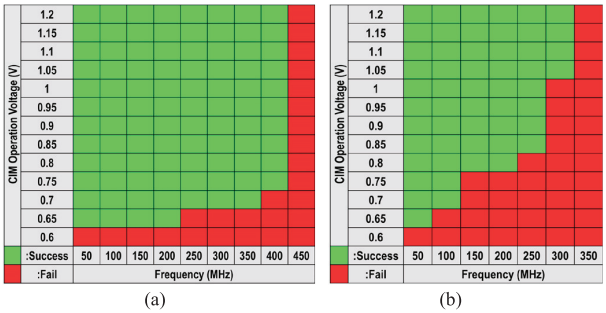


Fig. 17. Measured shmoo plots for (a) 1b–1b and (b) 4b–4b MAC operations.

weight was stored in one GMU, while the remaining 4-bit weight was allocated to a different GMU. The first four bits of the 8-bit input were fed into the macro, initiating a MAC operation between the 4-bit input and the 8-bit weight. After five cycles, including accumulation, the initial partial MAC result was obtained. Subsequently, the remaining 4-bit input was processed using the same five-cycle sequence, yielding the second partial MAC result. These partial results were then combined through external logic, completing the 8b–8b MAC operation in 11 cycles. The flexibility enabled the DCIM macro to perform INT8 MAC operations.

IV. Measurement Results

Fig. 15 presents the measurement setup and die micrograph of the prototype chip fabricated using a 28-nm CMOS process. A controller module that manages the input and output signals of the prototype chip was implemented on the FPGA board (DE-10 Standard). The chip was powered through a dc power supply (Keysight E3631A), and the power consumption was measured using a digital multimeter (Keysight 34550A).

Fig. 16(a) and (b) shows the area and power breakdowns of the proposed DMM-DCIM macro, respectively. The macro occupies an area of 0.017 mm^2, with the 16-kb DMM-DCIM subarray accounting for 82%, the drivers for 13%, the row peripheral circuits for 3%, and the column peripheral circuits for 7.2% [see Fig. 16(a)]. As shown in Fig. 16(b), the 4T1C

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.

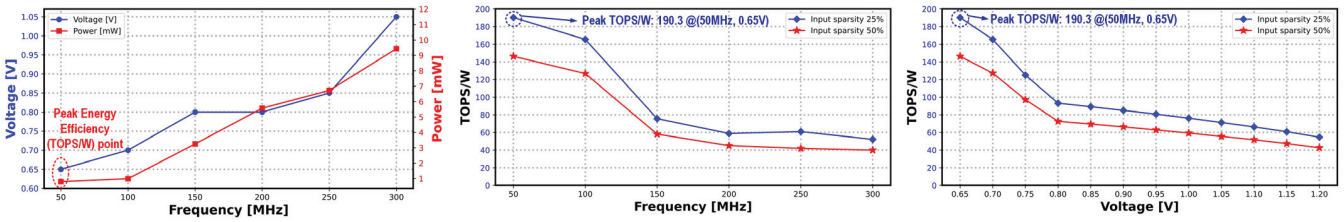


Fig. 18. Measured (a) voltage-frequency scaling plot, (b) TOPS/W versus frequency, and (c) supply voltage for the 4b–4b MAC operation.

TABLE I
PERFORMANCE COMPARISON WITH RECENT CIM DESIGNS

	This Work	[10]	[12]	[13]	[19]	[22]	[24]	[25]	[26]	[28]
Tech. (nm)	28	28	65	65	65	22	28	55	12	28
MAC Method	Digital	Analog	Analog	Analog	Analog	Digital	Digital	Digital	Digital	Analog
Memory Type	eDRAM	eDRAM	eDRAM	eDRAM	eDRAM	SRAM	SRAM	SRAM	SRAM	SRAM
Cell Structure	4T1C	3T2C	4T2C	2T1C	3T1C	6T	6T	[f] 6T + 10T	6T	10T3C
Array Size	16Kb	100Kb	8.5Kb	32Kb	8Kb	64Kb	16Kb	4Kb	8Kb	288Kb
Bit Cell Area (um^2)	**0.15**	0.95	[e] 0.22	N/A	[e] 0.31	[e] 0.611	N/A	[e, f] 1.09	N/A	0.756
Macro Area (mm^2)	**0.017**	[d] 0.029	[d, e] 0.022	[d, e] 0.156	[d, e] 0.074	[d, e] 0.081	0.049	[d, e] 0.065	[d, e] 0.347	[d] 0.073
DRT (µs)	57.4–211	40	350	N/A	360	-	-	-	-	-
Voltage (V)	0.65–1.2 (VDD) 0.65–0.8 (LVDD)	1	1.2	1	0.85–1.1	1	0.45–1.1	0.7–1.2	0.8	0.8
Max. Freq. (MHz)	400 (1b-1b) 300 (4b-4b)	250	250	200	105	100	250	100	800	200
Dual Multiply	**O (XNOR/AND)**	X	X	X	X	X	X	O	X	X
IN/W (b)	[a] 1–8/1–8	4, 8/5, 9	1–4/4	1–8/1,4,8	4, 8/4, 8	4–8/4,8	1–4/1	1–8/1–8	4–8/4, 8	4/2
Full Precision	O	X	X	X	X	O	X	O	O	X
TOPS/W	**517.6 (1b-1b)** [b] **190.3 (4b-4b)** [c] **52.9 (8b–8b)**	115.8 (4b-5b)	49.20 (4b-4b)	29.6 (4b-4b)	102.2 (4b-4b)	89 (4b-4b)	154 (@0.9V, 4b-1b)	44.82 (4b-4b)	121 (4b-4b)	[g] -
TOPS/mm^2	**873.8 (1b-1b)** **28.9 (4b-4b)** [c] **6.5 (8b–8b)**	2.03 (4b-5b)	[e] 19.04 (4b-4b)	[e] 0.26 (4b-4b)	[e] 44.96 (4b-4b)	[c] 10.13 (4b-4b)	98 (4b-1b)	[e] 3.19 (4b-4b)	[e] 7.86	[h] -

[a]: INT8 operation can be performed with external logic. [b]: Measured for input sparsity of 25%. [c]: Estimated result. [d]: Normalized for 16Kb array size. [e]: Normalized for 28-nm CMOS technology. [f]: 6T SRAM + multiplier (each 6T bit cell needs 1 multiplier). [g]: 6143–8161 TOPS/W/b. [h]: 111.8 TOPS/mm^2/b.

subarray consumes 62% of the total operating power of the macro.

Fig. 17 depicts the measured voltage-frequency shmoo plots of the proposed macro. In the 1b–1b AND/XNOR mode, the CIM operation was verified at 400 MHz above a VDD of 0.7 V. In the 4b–4b AND mode, the CIM operations up to 300 MHz were demonstrated at VDD levels exceeding 1.05 V. There are some discrepancies between two shmoo plots because, in the 1b–1b operation, the output of the local adder is the result of the MAC operation unlike the 4b–4b operation requires the calculation of a global adder and accumulator. Fig. 18 presents the measured results for the 4b–4b MAC operation, including the voltage-frequency scaling plot [see Fig. 18(a)], energy efficiency (in TOPS/W) versus frequency [see Fig. 18(b)], and energy efficiency versus supply voltage [see Fig. 18(c)]. A peak energy efficiency of 190.3 TOPS/W was achieved at 0.65 V and 50 MHz.

Table I compares the performances with recent CIM designs. The proposed CIM macro supports dual-multiplication modes based on the precharge state and enables reconfigurable input and weight-bit widths. Furthermore, it accommodates both signed and unsigned operations. While prior CIMs [25], [27] require more than six transistors per cell to support dual-mode multiplication, the proposed DMM-DCIM achieves this with only eight transistors per row for dual-mode multiplication. The proposed CIM macro achieved better energy and area efficiencies (TOPS/mm^2) compared to prior works while minimizing bit cell area and supporting XNOR and AND DMM.

V. CONCLUSION

The capacity of parameter processing using memory has expanded with the advancement of AI technology. Recent studies have focused on CIM architectures to address memory bottlenecks. Although DCIM offers superior AI accuracy compared to ACIM, improving its area and energy efficiencies is essential for edge device deployment. This work introduces a precharge-controlled 4T1C eDRAM-based DMM-DCIM macro with an AEA tree architecture. The proposed macro supports XNOR and AND operations through

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.

precharge state control, along with reconfigurable bit width and signed/unsigned functionality. These features make the macro suitable for various neural networks used in edge AI. Fabricated using a 28-nm CMOS process, the prototype chip demonstrated 190.3 TOPS/W and 28.9 TOPS/mm^2 in the 4b–4b MAC operation.

ACKNOWLEDGMENT

The chip fabrication and EDA tools were supported by the IC Design Education Center (IDEC), South Korea.

REFERENCES

[1] J.-H. Chae, "High-bandwidth and energy-efficient memory interfaces for the data-centric era: Recent advances, design challenges, and future prospects," *IEEE Open J. Solid-State Circuits Soc.*, vol. 4, pp. 252–264, 2024, doi: 10.1109/ojssc.2024.3458900.

[2] L. Wei et al., "13.3 a 7Mb STT-MRAM in 22FFL FinFET technology with 4ns read sensing time at 0.9 V using write-verify-write scheme and offset-cancellation sensing technique," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 214–216.

[3] Y.-D. Chih et al., "13.3 a 22nm 32Mb embedded STT-MRAM with 10ns read speed, 1M cycle write endurance, 10 years retention at 150°C and high immunity to magnetic field interference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 222–224.

[4] P. Jain et al., "13.2 a 3.6Mb 10.1Mb/mm^2 embedded non-volatile ReRAM macro in 22nm FinFET technology with adaptive forming/set/reset schemes yielding down to 0.5 V with sensing time of 5ns at 0.7 V," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2019, pp. 212–214.

[5] M.-F. Chang et al., "19.4 embedded 1Mb ReRAM in 28nm CMOS with 0.27-to-1 V read using swing-sample-and-couple sense amplifier and self-boost-write-termination scheme," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 332–333.

[6] V. Seshadri et al., "Ambit: In-memory accelerator for bulk bitwise operations using commodity DRAM technology," in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2017, pp. 273–287.

[7] S. Li, D. Niu, K. T. Malladi, H. Zheng, B. Brennan, and Y. Xie, "DRISA: A DRAM-based reconfigurable in-situ accelerator," in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2017, pp. 288–301.

[8] Z. Yue, Y. Wang, Y. Qin, L. Liu, S. Wei, and S. Yin, "BR-CIM: An efficient binary representation computation-in-memory design," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 10, pp. 3940–3953, Oct. 2022.

[9] I. Shah, K. Sarfraz, and M. Chan, "An efficient 6TP SRAM-based CIM macro with column ADCs for binarized neural networks," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 5, pp. 2509–2513, May 2024.

[10] S. Kim et al., "16.5 DynaPlasia: An eDRAM in-memory-computing-based reconfigurable spatial accelerator with triple-mode cell for dynamic resource switching," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 256–257.

[11] S. Xie, C. Ni, A. Sayal, P. Jain, F. Hamzaoglu, and J. P. Kulkarni, "EDRAM-CIM: Reconfigurable charge domain compute-in-memory design with embedded dynamic random access memory array realizing adaptive data converters," *IEEE J. Solid-State Circuits*, vol. 59, no. 6, pp. 1950–1961, Jun. 2024.

[12] I.-J. Jung, D. H. Kim, M. Jo, D. H. Ko, Y. Lee, and S.-O. Jung, "A charge-domain 4T2C eDRAM compute-in-memory macro with enhanced variation tolerance and low-overhead data conversion schemes," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 4, pp. 1824–1828, Apr. 2024.

[13] S. Xie, C. Ni, P. Jain, F. Hamzaoglu, and J. P. Kulkarni, "Gain-cell CIM: Leakage and bitline swing aware 2T1C gain-cell eDRAM compute in memory design with bitline precharge DACs and compact Schmitt Trigger ADCs," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 112–113.

[14] H. Yoon, D. Kim, G. Lee, and S. Cho, "A DRAM-based process-in-memory using data redundancy and differential bit-line computation," *IDEC J. Integr. Circuits Syst.*, vol. 10, no. 1, pp. 5–10, Jan. 2024.

[15] S. Zhu, L. H. K. Duong, and W. Liu, "XOR-Net: An efficient computation pipeline for binary neural network inference on edge devices," in *Proc. IEEE 26th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Dec. 2020, pp. 124–131.

[16] P. Srivastava et al., "PROMISE: An end-to-end design of a programmable mixed-signal accelerator for machine-learning algorithms," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2018, pp. 43–56.

[17] J. Cho and W. Jung, "Hardware-software co-design for analog compute-in-memory accelerators using 1-bit sense amplifiers," *IDEC J. Integr. Circuits Syst.*, vol. 11, no. 1, pp. 44–48, Jan. 2025.

[18] S.-E. Hsieh et al., "7.6 A 70.85-86.27TOPS/W PVT-insensitive 8b word-wise ACIM with post-processing relaxation," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 136–137.

[19] Z. Chen, X. Chen, and J. Gu, "A 65nm 3T dynamic analog RAM-based computing-in-memory macro and CNN accelerator with retention enhancement, adaptive analog sparsity and 44TOPS/W system energy efficiency," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 240–241.

[20] D. Wang, C.-T. Lin, G. K. Chen, P. Knag, R. K. Krishnamurthy, and M. Seok, "DIMC: 2219TOPS/W 2569F2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 266–268.

[21] J. Oh, C.-T. Lin, and M. Seok, "D6CIM: 60.4-TOPS/W, 1.46-TOPS/mm^2, 1005-Kb/mm^2 digital 6T-SRAM-based compute-in-memory macro supporting 1-to-8b fixed-point arithmetic in 28-nm CMOS," in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2023, pp. 413–416.

[22] Y.-D. Chih et al., "16.4 an 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 252–253.

[23] Y. Zhan, W.-H. Yu, K.-F. Un, R. P. Martins, and P.-I. Mak, "A 28-nm 18.7 TOPS/mm^2 89.4-to-234.6 TOPS/W 8b single-finger eDRAM compute-in-memory macro with bit-wise sparsity aware and kernel-wise weight update/refresh," *IEEE J. Solid-State Circuits*, vol. 59, no. 11, pp. 3866–3876, Nov. 2024.

[24] C.-T. Lin et al., "DIMCA: An area-efficient digital in-memory computing macro featuring approximate arithmetic hardware in 28 nm," *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 960–971, Mar. 2024.

[25] H. You, W. Li, D. Shang, Y. Zhou, and S. Qiao, "A 1–8b reconfigurable digital SRAM compute-in-memory macro for processing neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 4, pp. 1602–1614, Apr. 2024.

[26] C.-F. Lee et al., "A 12nm 121-TOPS/W 41.6-TOPS/mm^2 all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 24–25.

[27] H. Jia, H. Valavi, Y. Tang, J. Zhang, and N. Verma, "A programmable heterogeneous microprocessor based on bit-scalable in-memory computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 9, pp. 2609–2621, Sep. 2020.

[28] J. Lee, B. Zhang, and N. Verma, "A switched-capacitor SRAM in-memory computing macro with high-precision, high-efficiency differential architecture," in *Proc. IEEE Eur. Solid-State Electron. Res. Conf. (ESSERC)*, Sep. 2024, pp. 357–360.



Ho-Sung Lee received the B.S. degree in electronics communications engineering from Kwangwoon University, Seoul, South Korea, in 2023, where he is currently working toward the Ph.D. degree at the Department of Electronics and Communications Engineering.

His research interests include integrated circuit design for AI accelerators such as dynamic random access memory (DRAM), static RAM (SRAM), and embedded DRAM (eDRAM)-based processing-in-memory design.

Mr. Lee received the Corporate Special Award from the 24th Korea Semiconductor Design Competition in 2023 and the Bronze Prize (Best Paper Award) at the International Conference on Electronics, Information, and Communication (ICEIC) in Taiwan in 2024.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.



Ik-Hyeon Jeon received the B.S. degree in electronic convergence engineering from Kwangwoon University, Seoul, South Korea, in 2024, where he is currently working toward the M.S. degree at the Department of Electronics and Communications Engineering.

His research interests include integrated circuits and compute-in-memory design.

Mr. Jeon received the Minister of Science and ICT Award from the Artificial Intelligence Semiconductor Idea Competition in 2023.



Jiwon Lee received the B.S. degree in electronics and communications engineering from Kwangwoon University, Seoul, South Korea, in 2023, where she is currently working toward the M.S. degree.

Her research interests include mixed-signal integrated circuits and processing-in-memory design.



Eun-Bi Koh received the B.S. degree in electronics and information engineering from Sejong University, Seoul, South Korea, in 2024. She is currently working toward the integrated M.S. and Ph.D. degrees at the Department of Electronics and Communications Engineering, Kwangwoon University, Seoul.

Her research interests include mixed-signal integrated circuits and compute-in-memory design.



Joo-Hyung Chae (Senior Member, IEEE) received the B.S. and Ph.D. degrees in electrical engineering from Seoul National University, Seoul, South Korea, in 2012 and 2019, respectively.

In 2013, he joined the Department of LPDDR Memory Design, SK hynix, Icheon, South Korea, as an Intern. From 2019 to 2021, he was with SK hynix, where his work focused on graphics double data rate (GDDR) memory design. In 2021, he joined Kwangwoon University, Seoul, where he is currently an Assistant Professor of Electronics and Communications Engineering. His research interests include the design of high-speed and low-power I/O circuits, clocking circuits, memory interfaces, and mixed-signal in-memory computing.

Dr. Chae received the Doyeon Academic Paper Award from the Inter-University Semiconductor Center (ISRC) of Seoul National University in 2020.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:40:12 UTC from IEEE Xplore. Restrictions apply.